

FIL FRESHER BOOTCAMP · DAY PLAN

Engineering Practices That Ship

Standards · TDD · Refactoring · Pairing · Inner Source · Terraform Modules

A hands-on, code-first day: 6 labs + an end-of-day capstone



Standards



TDD



Refactor



Pairing



Inner source



Terraform

Facilitator: Abinash Kumar Mishra

The day at a glance

Every module ends with a lab; the capstone uses all six practices together

09:30	●	Kick-off & laptop setup
09:45	●	M1 Code & repository standards
11:00	●	M2 Test-driven development
12:15	●	M3 Refactoring
13:45	●	M4 Pair programming
14:30	●	M5 Inner source engineering at FIL
15:30	●	M6 Terraform inner-source modules
16:30	●	Capstone build (pairs)
17:45	●	Capstone demos & retro

Lab 1

Lab 2

Lab 3

Lab 4

Lab 5

Lab 6

75 min

30 min

Breaks

- 10:45 – 11:00 Tea
- 13:00 – 13:45 Lunch
- 15:15 – 15:30 Tea

Time split

~70%

of the day is hands-on keyboard time

Before we start: your lab environment

Get to green in 10 minutes. Raise a hand if any command fails.

terminal

```
# 1. clone the bootcamp repo (internal Git server)
$ git clone <internal-git-url>/fil-bootcamp-labs.git
$ cd fil-bootcamp-labs

# 2. Python env + tools (via internal package mirror)
$ python -m venv .venv && source .venv/bin/activate
$ pip install pytest ruff pre-commit

# 3. hooks + first test run
$ pre-commit install
$ pytest # expect: all tests pass
$ terraform -version # expect: 1.7 or newer
```



Python 3.10+, Git, VS Code

with the Python & HashiCorp Terraform extensions



Terraform 1.7+ and tfint

`terraform test` with mock providers needs 1.7+



A pairing partner

you will switch partners after lunch

01



Code & Repository Standards

09:45 – 10:45

BY THE END OF THIS MODULE YOU CAN



Explain why standards matter more in a regulated bank



Apply naming, typing and formatting rules automatically



Set up a repo with the files & guardrails reviewers expect

Why standards? Code is read far more than written

M1 · STANDARDS



Readability

Any engineer can pick up any repo. On-call at 2 a.m. is not the time to decode clever code.



Faster reviews

Reviewers focus on logic, not tabs vs spaces. Tools settle style before a human looks.



Audit & compliance

In a regulated firm every change must be traceable to a ticket, a reviewer and a passing pipeline.



Security by default

Secret scanning, dependency checks and SAST catch problems before they reach main.

Code standards: before and after

M1 · STANDARDS

Same logic. One version you can review in 10 seconds.

fee.py

```
# ❌ before
def calc(a,b,c=False):
    x=a*0.0025
    if c==True: x=x*0.5
    if x<10: x=10
    return round(x,2)

pwd = "Summer2026!"    # hard-coded secret
```

fee_calculator.py

```
# ✅ after
MIN_FEE = Decimal("10.00")
RATE = Decimal("0.0025")    # 25 bps

def calculate_fee(trade_value: Decimal,
                  is_premium: bool = False) -> Decimal:
    """Return the brokerage fee, rounded to 2 dp."""
    fee = trade_value * RATE
    if is_premium:
        fee *= Decimal("0.5")
    return max(fee, MIN_FEE).quantize(Decimal("0.01"))
```

- **Names** reveal intent: calculate_fee, not calc
- **No magic numbers** named constants
- **Types + docstring** on every public function
- **Decimal for money** never float
- **Secrets** come from a vault, never code

Style (PEP 8, import order, line length) is enforced by ruff, not by reviewers.

A standard repository layout

If every repo looks the same, onboarding takes minutes, not days

M1 · STANDARDS

```
fil-bootcamp-labs/
├── README.md           # what, why, how to run
├── CONTRIBUTING.md     # how to contribute (inner source)
├── CODEOWNERS          # who must review what
├── CHANGELOG.md        # user-visible changes per version
├── pyproject.toml      # deps + ruff + pytest config
├── .pre-commit-config.yaml # local quality gates
├── .github/
│   └── PULL_REQUEST_TEMPLATE/
├── src/
│   └── fees/           # production code
├── tests/              # mirrors src/
├── tf/
│   └── modules/        # terraform modules
```



Docs live with code

README and CONTRIBUTING are required; a repo without them is not 'done'.



tests/ mirrors src/

src/fees/fee_calculator.py →
tests/test_fee_calculator.py



Ownership is explicit

CODEOWNERS auto-requests the right reviewers on every PR.

Branches, commits and pull requests

M1 · STANDARDS



terminal

```
# short-lived branch named after the ticket
$ git switch -c feature/PAY-142-premium-discount

# Conventional Commits: type(scope): summary [ticket]
$ git commit -m "feat(fees): add premium discount [PAY-142]"
$ git commit -m "fix(fees): round half-up to 2 dp [PAY-151]"
$ git commit -m "test(fees): cover minimum-fee boundary"

$ git push -u origin feature/PAY-142-premium-discount
```

- **Trunk-based:** branches live hours or days, not weeks
- **Small PRs:** under ~400 changed lines
- **Types:** feat, fix, test, refactor, docs, chore
- **Every PR:** linked ticket, green CI, CODEOWNER approval
- **Never:** force-push to main or commit secrets

Ticket keys (PAY-142) are illustrative. Use your team's JIRA project key.

FIL-aligned guardrails: what the pipeline enforces

M1 · STANDARDS

Checklist to confirm against the FIL Engineering Handbook for your team



Protected main

No direct pushes. Merge via PR only; branch must be up to date.



Mandatory review

≥1 CODEOWNER approval (2 for prod infra); author can't self-approve.



Security scans

Secret scanning, dependency (SCA) and static analysis (SAST) on every PR.



Quality gate

Tests green, coverage must not drop, lint clean. Red build = no merge.



Traceability

Ticket reference in branch/commit/PR so every change maps to a change record.



Approved sources

Dependencies and modules only from internal mirrors / registry.

Make the repo reviewer-ready

- 1 **Add quality hooks** copy the pre-commit config on the right
- 2 **Run on everything** pre-commit run --all-files and fix what fails
- 3 **Fix messy_fee.py** rename, add types, docstring, constants
- 4 **Add CODEOWNERS + PR template** use the layout from the previous slide
- 5 **Commit properly** feat/chore with a ticket key; open a PR

Done when: pre-commit passes and the PR shows CODEOWNERS as reviewers.

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.6.9
  hooks:
    - id: ruff          # lint
      args: [--fix]
    - id: ruff-format   # format
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.21.2
  hooks:
    - id: gitleaks      # block committed secrets
- repo: https://github.com/antonbabenko/pre-commit-terraform
  rev: v1.96.1
  hooks:
    - id: terraform_fmt
    - id: terraform_tflint
```

02



Test-Driven Development

11:00 – 12:15

BY THE END OF THIS MODULE YOU CAN



Run the red → green → refactor loop in small steps



Write clear Arrange-Act-Assert tests with pytest



Isolate slow or external dependencies with mocks

The TDD loop: red → green → refactor

Tests first means design first: you decide how the code is used before how it works

1 RED

Write a small test that fails

Proves the test can fail and pins down the next behaviour.

2 GREEN

Write just enough code to pass

Resist the urge to build the whole feature. Fake it if needed.

3 REFACTOR

Clean up with tests green

Remove duplication, improve names. Behaviour stays the same.



Loop length: 2–5 minutes. If you've been red for 10 minutes, your step is too big. Delete, and take a smaller one.

Anatomy of a good test: Arrange · Act · Assert

```
tests/test_fee_calculator.py

from decimal import Decimal
from src.fees.fee_calculator import calculate_fee

def test_fee_is_25_bps_of_trade_value():
    # Arrange
    trade_value = Decimal("100000")

    # Act
    fee = calculate_fee(trade_value)

    # Assert
    assert fee == Decimal("250.00")
```

- **Name = spec.** Read test names aloud; they should describe the behaviour.
- **One behaviour per test.** When it fails you know exactly what broke.
- **Fast & isolated.** No network, no DB, no clock. Milliseconds, not seconds.
- **Deterministic.** Same result on every run, on every machine.
- **F.I.R.S.T.** Fast · Independent · Repeatable · Self-validating · Timely

```
$ pytest -q
E ModuleNotFoundError: src.fees.fee_calculator ← RED ✓
```

Green, then grow the spec with parametrised tests

M2 · TDD

src/fees/fee_calculator.py

```
MIN_FEE = Decimal("10.00")
RATE = Decimal("0.0025") # 25 bps

def calculate_fee(trade_value: Decimal,
                 is_premium: bool = False) -> Decimal:
    if trade_value <= 0:
        raise ValueError("trade_value must be positive")
    fee = trade_value * RATE
    if is_premium:
        fee = fee * Decimal("0.5")
    fee = max(fee, MIN_FEE)
    return fee.quantize(Decimal("0.01"),
                       rounding=ROUND_HALF_UP)
```

tests/test_fee_calculator.py

```
@pytest.mark.parametrize(
    "value, premium, expected",
    [
        (Decimal("1000"), False, Decimal("10.00")),
        (Decimal("100000"), True, Decimal("125.00")),
        (Decimal("5000.20"), False, Decimal("12.50")),
    ],
)
def test_fee_rules(value, premium, expected):
    assert calculate_fee(value, is_premium=premium) \
        == expected

def test_rejects_non_positive_trade_value():
    with pytest.raises(ValueError):
        calculate_fee(Decimal("0"))
```

Each row was added as its own red → green cycle: minimum fee, then premium, then rounding, then validation.

Isolate dependencies with test doubles

Your unit test must not call a live FX service, database or clock

`src/fees/fx_converter.py`

```
class FxConverter:
    def __init__(self, provider: RateProvider):
        self._provider = provider # injected

    def convert(self, amount, base, quote):
        if base == quote:
            return amount
        rate = self._provider.get_rate(base, quote)
        return (amount * rate).quantize(Decimal("0.01"))
```

`tests/test_fx_converter.py`

```
from unittest.mock import Mock

def test_converts_using_provider_rate():
    provider = Mock()
    provider.get_rate.return_value = Decimal("83.25")
    fx = FxConverter(provider)

    assert fx.convert(Decimal("100"), "USD", "INR") \
        == Decimal("8325.00")
    provider.get_rate.assert_called_once_with("USD", "INR")
```

Stub

returns canned data

Mock

also verifies it was called correctly

Fake

a working lightweight version (in-memory repo)

How much testing, and where?

M2 · TDD

E2E / UI

few · slow · brittle

Integration / contract

some · real DB, API contracts

Unit

many · milliseconds · written with TDD

Coverage: a smoke alarm, not a target

- Use it to find untested code, not to prove quality
- Rule of thumb: coverage must not go down in a PR
- 100% coverage with no asserts tests nothing
- Mutation testing (mutmut, PIT) checks test strength

```
$ pytest --cov=src --cov-report=term-missing
```

TDD kata: build the trade fee calculator

Rules: no production code without a failing test. Commit on every green.

- 1 **Fee is 25 bps of trade value** 100,000 → 250.00
- 2 **Minimum fee is 10.00** 1,000 → 10.00
- 3 **Premium clients get 50% off** applied before the minimum
- 4 **Round half-up to 2 dp** 5,000.20 → 12.50
- 5 **Reject value ≤ 0** raise ValueError
- 6 **Stretch: FX conversion** fee in INR via a mocked RateProvider

terminal

```
# the loop, every 2-5 minutes
$ pytest -q                # RED: 1 failed
# ... write the minimum code ...
$ pytest -q                # GREEN: all passed
$ git commit -am "feat(fees): minimum fee"
# ... tidy names / duplication ...
$ pytest -q                # still GREEN
$ git commit -am "refactor(fees): extract RATE"
```

Done when: 6 behaviours, 6+ tests, 10+ small commits.

03



Refactoring

12:15 – 13:00

BY THE END OF THIS MODULE YOU CAN



Spot common code smells in legacy code



Lock behaviour with a characterization test



Apply named refactorings in small, safe steps

Refactoring = change structure, keep behaviour

M3 · REFACTORING

Smells tell you where; named refactorings tell you how

Smell	What you see	Refactoring to apply
Long function	One function does parsing, math and formatting	Extract Function
Cryptic names	t, f, o, x, process()	Rename Variable / Function
Magic numbers	0.0025, 0.0010, 5.0 inline	Replace Magic Number with Constant
Growing if/elif	New trade type = edit the same chain	Replace Conditional with Lookup / Polymorphism
Primitive obsession	Dicts with string keys everywhere	Introduce Parameter Object / dataclass
Duplicated logic	qty * px computed in 3 places	Extract Method / Property

Golden rule: never refactor and change behaviour in the same commit.

Legacy code: find the smells

refactor/report_before.py: it works, it's in production, nobody wants to touch it

`refactor/report_before.py`

```
def process(orders):
    t = 0
    out = []
    for o in orders:
        if o["type"] == "EQ":
            f = o["qty"] * o["px"] * 0.0025
        elif o["type"] == "FI":
            f = o["qty"] * o["px"] * 0.0010
        elif o["type"] == "FX":
            f = 5.0
        else:
            f = 0
        if f < 1:
            f = 1
        t = t + o["qty"] * o["px"] + f
        out.append(o["id"] + "," + str(round(f, 2)))
    out.append("TOTAL," + str(round(t, 2)))
    return "\n".join(out)
```

Pair challenge (3 min)

How many smells can you find? Name each one using the table from the previous slide.

- Cryptic names: t, f, o
- Magic numbers
- if/elif chain per trade type
- qty * px duplicated
- Dicts instead of a type
- Computing and formatting mixed

Step 0: lock behaviour with a characterization test

You don't need to know if the output is right, only that it doesn't change

```
tests/test_refactor_characterization.py
```

```
from refactor.report_before import process
from refactor.report_after import Order, build_report

RAW = [
    {"id": "T1", "type": "EQ", "qty": 100, "px": 250.0},
    {"id": "T2", "type": "FI", "qty": 10, "px": 99.5},
    {"id": "T3", "type": "FX", "qty": 1, "px": 1000.0},
    {"id": "T4", "type": "XX", "qty": 1, "px": 10.0},
]

def test_refactored_report_matches_legacy_output():
    legacy = process(RAW)
    modern = build_report([Order(**o) for o in RAW])
    assert modern == legacy
```



Real catch from building this lab

```
- T2,1
+ T2,1.0
```

The refactor used 1.0 as the minimum fee; legacy used the integer 1. A downstream file parser would have seen a changed format. The characterization test caught it in milliseconds.

After: same output, readable structure

M3 · REFACTORING

refactor/report_after.py (1/2)

```
FEE_RATES = {"EQ": 0.0025, "FI": 0.0010}
FLAT_FEES = {"FX": 5.0}
MIN_FEE = 1 # int on purpose: legacy prints "1"
```

```
@dataclass(frozen=True)
```

```
class Order:
    id: str
    type: str
    qty: int
    px: float
```

```
@property
```

```
def notional(self) -> float:
    return self.qty * self.px
```

refactor/report_after.py (2/2)

```
def fee_for(order: Order) -> float:
    if order.type in FLAT_FEES:
        fee = FLAT_FEES[order.type]
    else:
        fee = order.notional * FEE_RATES.get(order.type, 0.0)
    return max(fee, MIN_FEE)
```

```
def build_report(orders: list[Order]) -> str:
    lines = [f"{o.id},{round(fee_for(o), 2)}" for o in orders]
    total = sum(o.notional + fee_for(o) for o in orders)
    lines.append(f"TOTAL,{round(total, 2)}")
    return "\n".join(lines)
```

Introduce dataclass

Extract property

Constants → lookup table

Extract function

Rename

Refactor the legacy report safely

1

Run the characterization test green against report_before.py

2

Rename t, f, o IDE rename (F2), run tests, commit

3

Introduce the Order dataclass run tests, commit

4

Extract fee_for() and notional run tests, commit

5

Replace if/elif with lookup tables run tests, commit

6

Stretch: add trade type 'MF' at 15 bps now a one-line change, test-first

Safe refactoring rules

- **Green before, green after.** Never start from red.
- **Tiny steps.** If tests break, undo; don't debug.
- **Let the IDE do it.** Automated rename/extract is safer than typing.
- **Commit every step.** refactor: prefix, no behaviour change.
- **Stop when it's clean enough.** Not perfect; better than you found it.

04



Pair Programming

13:45 – 14:30

BY THE END OF THIS MODULE YOU CAN



Pair using driver/navigator and ping-pong styles



Rotate roles on a timer and communicate while coding



Pair remotely with shared editors

Three ways to pair

Two people, one problem, one keyboard at a time



Driver / Navigator

Driver types and thinks tactically.
Navigator reviews each line, thinks ahead
and keeps the list of next tests.

Swap every 10–15 min



Ping-pong (TDD)

A writes a failing test. B makes it pass and
writes the next failing test. Repeat.

Swap on every test



Strong-style

"For an idea to go into the computer, it
must go through someone else's hands."
The navigator decides; the driver types.

Great for onboarding

Pairing is not one person coding while the other watches. If the navigator is silent, swap.

Pairing etiquette: what good pairs say



Say this

- "What test should we write next?"
- "Can you walk me through that line?"
- "Let's try your idea first, then compare."
- "I'm lost. Can we step back?"



Not this

- "Just give me the keyboard."
- "That's wrong." (without why)
- Silently checking your phone
- Typing without talking

Remote pairing kit

- VS Code Live Share (or approved equivalent)
- Teams call with camera on
- Shared timer for role swaps
- git mob / Co-authored-by trailer for credit

```
$ git commit -m "feat(auth): password policy"
```

```
Co-authored-by: Priya S <priya.s@example.com>
```

Ping-pong TDD: password policy validator

New partner. Person A writes the failing test, B makes it pass and writes the next test. Swap every test.

- 1 At least 12 characters
- 2 Contains an uppercase letter
- 3 Contains a digit
- 4 Contains a special character
- 5 Return ALL unmet rules, not just the first
- 6 Refactor to a rules table together, tests green

```
# tests/test_password_policy.py (A writes first)
def test_short_password_reports_length():
    assert "at least 12 characters" in validate("Ab1!")

# pairing/password_policy.py (after refactor)
RULES = [
    (lambda p: len(p) >= 12, "at least 12 characters"),
    (lambda p: re.search(r"[A-Z]", p), "an uppercase letter"),
    (lambda p: re.search(r"\d", p), "a digit"),
    (lambda p: re.search(r"[^\w\s]", p), "a special character"),
]

def validate(password: str) -> list[str]:
    return [msg for rule, msg in RULES if not rule(password)]
```

05



Inner Source Engineering at FIL

14:30 – 15:15

BY THE END OF THIS MODULE YOU CAN



Explain inner source and why FIL uses it



Know the roles: owner, trusted committer, contributor



Make a first contribution to another team's repo

Inner source: open-source ways of working, inside the firm

M5 · INNER SOURCE

Code stays private to FIL, but anyone in FIL can read it, reuse it and contribute to it



Reuse over rebuild

One shared library or module instead of five teams writing their own.



Unblock yourself

Need a change in another team's service? Raise a PR instead of waiting in their backlog.



Better quality

More eyes on the code; security fixes land once and everyone benefits.



Build knowledge

Transparent code, docs and discussions help teams learn from each other.

Open source vs inner source

	Open source	Inner source
Who can see	Anyone	FIL employees
Who can contribute	Anyone	Any FIL engineer
Governance	Foundation / maintainers	Owning team + trusted committers
Compliance	Licences	FIL policies, change control

Roles and the contribution flow

M5 · INNER SOURCE

Repo owner

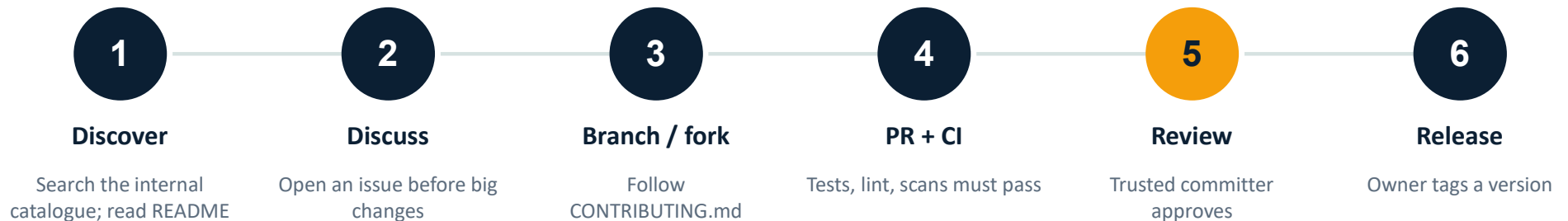
Owning team. Sets direction, roadmap and standards.

Trusted committer

Reviews and merges PRs, mentors contributors.

Contributor

Any engineer. Opens issues and PRs.



Contributor etiquette: follow the host repo's standards, not yours · keep PRs small · respond to review within a day · write the docs for what you add.

The files that make a repo inner-source ready

M5 · INNER SOURCE

CONTRIBUTING.md

```
# Contributing

1. Open (or pick) an issue and comment that you are taking it.
2. Branch from `main`: `feature/<JIRA-KEY>-short-desc`.
3. Write a failing test first, then the code. `pytest` must be green.
4. Run `pre-commit run --all-files` before pushing.
5. Commit using Conventional Commits:
   `feat(fees): add premium discount [JIRA-123]`
6. Open a PR using the template; at least 1 trusted committer
   must approve.
7. Keep PRs under ~400 changed lines. Update CHANGELOG.md.
```



README.md

What it does, quick start, who owns it



CONTRIBUTING.md

How to contribute, step by step



CODEOWNERS

Auto-assigns trusted committers



CHANGELOG.md

What changed in each version



Issue / PR templates

Consistent, reviewable requests

```
/tf/modules/ @fil-bootcamp/platform-engineering
```

Your first inner-source contribution

Each pair 'owns' their repo from Labs 1–4. Swap repo links with the pair next to you and contribute to theirs.

- 1 Read their README + CONTRIBUTING** can you run their tests in < 5 min? If not, that's issue #1
- 2 Open an issue** e.g. 'add trade type MF' or 'README missing setup step'
- 3 Branch, test-first change, PR** follow THEIR conventions and PR template
- 4 Review a PR on your own repo** as trusted committer: approve or request changes
- 5 Merge and update CHANGELOG** owner tags v0.2.0

```
# Review checklist for trusted committers
[ ] Linked issue and clear description
[ ] Tests added; CI green
[ ] Follows repo standards (lint, naming)
[ ] No secrets or client data
[ ] Docs / CHANGELOG updated
[ ] Small enough to review in 15 min
```

```
# Comment style
"nit:"  optional polish
"q:"    a question, not a blocker
"must:" blocks merge; explain why
```

06



Terraform Inner-Source Modules

15:30 – 16:30

BY THE END OF THIS MODULE YOU CAN



Build a reusable, validated Terraform module



Version and consume it from an internal registry



Test it with terraform test and static checks

Why shared modules? Build the secure path once

Every team needs an S3 bucket. Not every team should decide how to encrypt it.

standard module layout

```
modules/s3-secure-bucket/  
├── README.md           # inputs, outputs, example  
├── CHANGELOG.md  
├── versions.tf         # terraform + provider pins  
├── variables.tf        # typed + validated inputs  
├── main.tf             # the resources  
├── outputs.tf          # what consumers can use  
├── examples/basic/     # runnable usage  
└── tests/  
    └── naming.tftest.hcl
```



Secure by default

Encryption, versioning and public-access block are built in; consumers can't forget them.



Consistent

Same naming and tags across all teams, so cost and ownership reports just work.



Faster delivery

A new bucket is 8 lines of code and an already-approved pattern.

Module = a product. It has owners, docs, versions and tests, just like application code.

Inside the module: validated inputs, safe resources

M6 • TERRAFORM

variables.tf

```
variable "environment" {
  description = "dev | uat | prod"
  type        = string
  validation {
    condition = contains(["dev", "uat", "prod"],
                        var.environment)
    error_message = "environment must be dev, uat or prod."
  }
}

variable "name" {
  type = string
  validation {
    condition = can(regex("^[a-z0-9-]{3,40}$", var.name))
    error_message = "3-40 chars: lowercase, digits, hyphens."
  }
}
```

main.tf

```
locals {
  bucket_name = "${var.team}-${var.environment}-${var.name}"
  tags = merge(var.tags, {
    Team = var.team, Environment = var.environment
    ManagedBy = "terraform", Module = "s3-secure-bucket"
  })
}

resource "aws_s3_bucket" "this" {
  bucket = local.bucket_name
  tags    = local.tags
}

resource "aws_s3_bucket_public_access_block" "this" {
  bucket = aws_s3_bucket.this.id
  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls    = true
  restrict_public_buckets = true
}

# + SSE-KMS encryption and versioning resources
```

Consuming and versioning the module

Semantic versioning tells consumers whether an upgrade is safe

`examples/basic/main.tf`

```
module "reports_bucket" {  
  # internal registry (example path)  
  source = "app.terraform.io/fil-platform/s3-secure-bucket/aws"  
  version = "~> 1.2"    # any 1.x from 1.2 up, never 2.0  
  
  name      = "trade-reports"  
  team      = "payments"  
  environment = "dev"  
  kms_key_arn = var.kms_key_arn  
}  
  
output "bucket" {  
  value = module.reports_bucket.bucket_arn  
}
```

MAJOR

2.0.0

Breaking: renamed input, resource replaced

MINOR

1.3.0

New optional input or feature

PATCH

1.2.1

Bug fix, no interface change

```
$ git tag v1.3.0 && git push origin v1.3.0
```

Never point prod at a branch or 'latest'. Pin a version, upgrade deliberately, read the CHANGELOG.

terraform test: TDD for infrastructure

tests/naming.tftest.hcl

```
mock_provider "aws" {} # no cloud credentials needed

variables {
  name      = "trade-reports"
  team      = "payments"
  environment = "dev"
  kms_key_arn = "arn:aws:kms:eu-west-1:111122223333:key/test"
}

run "bucket_name_follows_convention" {
  command = plan
  assert {
    condition     = aws_s3_bucket.this.bucket == "payments-dev-trade-reports"
    error_message = "Bucket name must be <team>-<env>-<name>."
  }
}

run "rejects_invalid_environment" {
  command = plan
  variables { environment = "production" }
  expect_failures = [var.environment]
}
```

Quality gates, in order

- 1 **terraform fmt -check**
formatting
- 2 **terraform validate**
syntax & types
- 3 **tflint**
provider best practice
- 4 **checkov / trivy**
security misconfig
- 5 **terraform test**
behaviour

Build, test and publish s3-secure-bucket

1

Write the failing test first naming.tftest.hcl → terraform test = RED

2

Implement variables.tf + main.tf until terraform test is GREEN

3

Add a validation + its test reject names with uppercase or '_'

4

Run all gates fmt, validate, tflint, checkov

5

Write README + CHANGELOG inputs, outputs, example usage

6

Tag v1.0.0 consume it from examples/basic

terminal

```
$ cd tf/modules/s3-secure-bucket
$ terraform init -backend=false
$ terraform test
tests/naming.tftest.hcl... in progress
  run "bucket_name_follows_convention"... pass
  run "mandatory_tags_present"... pass
  run "rejects_invalid_environment"... pass
tests/naming.tftest.hcl... pass

Success! 3 passed, 0 failed.
```

Stretch: add an optional lifecycle rule input (MINOR bump → v1.1.0).

CAPSTONE · 16:30 – 18:15

Trade Fee Service + its infrastructure

In pairs, ship a small but production-grade inner-source service using all six practices. Another pair is your 'consumer team' and must contribute one change.

New requirement (revealed at 17:15): premium clients get the FX fee waived.

Tests that let you change safely are the point.



Service

fees package: equities, fixed income, FX, MF fee rules + FX conversion



Infra

Terraform: your s3-secure-bucket module v1.x for fee reports



Inner source

README, CONTRIBUTING, CODEOWNERS; accept 1 PR from another pair



Way of working

Ping-pong TDD, swap every test, Co-authored-by on commits

Capstone plan and deliverables

CAPSTONE

105 minutes: build, contribute, change, demo

16:30	Set up	Repo from template, hooks, CODEOWNERS, first red test	15m
16:45	Build with TDD	Fee rules + FX conversion (mocked). Commit on every green.	30m
17:15	Change arrives	Premium FX waiver + refactor fee rules into a lookup	15m
17:30	Inner source + infra	Accept 1 PR from another pair; wire the bucket module v1.x	15m
17:45	Demo & retro	3 min per pair: tests, git log, PR thread, terraform test	30m

Definition of done

- ✓ pytest green, ≥ 15 tests
- ✓ pre-commit clean
- ✓ ≥ 15 small commits, co-authored
- ✓ 1 merged PR from another pair
- ✓ terraform test passes
- ✓ README lets a stranger run it in 5 min

How the capstone is scored

Scored on practices, not just on working code

CAPSTONE

Criterion	What we look for	Pts
Test-driven	Tests before code in git log; clear AAA test names; mocks for FX	25
Code & repo standards	Lint clean, typed, Decimal for money, standard layout, PR template	20
Refactoring	Change request absorbed via lookup/extract; separate refactor commits	15
Pair programming	Alternating authors / co-author trailers; both can explain the code	10
Inner source	CONTRIBUTING + CODEOWNERS; external PR reviewed & merged professionally	15
Terraform module	Validated inputs, pinned version, terraform test green	15

100

points total

- 80+ Ready for team work
- 60–79 Good, one gap to close
- < 60 Pair with a mentor next week

Take it back to your team

Standards

Let tools enforce style; spend reviews on design.

TDD

Red → green → refactor in 2–5 minute loops.

Refactoring

Characterize first, then tiny safe steps.

Pairing

Swap roles often; think out loud.

Inner source

Fix it yourself via PR; follow the host's rules.

Terraform

Modules are products: validate, test, version.

Next week: make one inner-source contribution to a real FIL repo, and pair on it.

Further reading: Beck, TDD by Example · Fowler, Refactoring (2nd ed.) · InnerSource Commons patterns · developer.hashicorp.com/terraform/language/tests